

THE CATHOLIC UNIVERSITY OF AMERICA

DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Programmable Logic Devices and HDL Design

PROJECT - 1

Designing Simple 4-bit Calculator

Submitted to: Professor. Thomas Kim

Student Name: Muluaem Ayena

Submitted On: 12 Oct 2025

Table of Contents

1. Purpose
2. Functional Specifications
3. Design Approach
4. Architecture Description
5. Hardware and Software Interaction and Executions
6. Functional block diagram
7. Schematics/Photos
8. Conclusions / Lessons Learned
9. Appendix / Source Code Listing

1. Purpose

The purpose of this project is to design and implement a **Simple 4-Bit Calculator** using **VHDL** on the **DE2-115 FPGA development board**.

The system performs basic arithmetic operations addition, subtraction, multiplication, and division on two signed 4-bit operands. It accepts input through on-board switches, decodes and displays operands and results using seven-segment displays, and includes proper error handling for invalid or overflow cases.

The objective is to demonstrate proficiency in:

- VHDL structural and behavioral modeling
- FPGA synthesis and simulation using **Intel Quartus Prime** and **ModelSim**
- Digital arithmetic operations and 7-segment display interfacing
- Modular design and verification through simulation

Problem Statements

The primary technical challenge involves designing a reliable digital system that can:

1. Accept user inputs through physical switches and buttons
2. Validate input operands for permissible ranges (0-15 decimal)
3. Perform four distinct arithmetic operations with correct signed arithmetic
4. Handle exceptional conditions such as division by zero
5. Convert binary results to decimal format for human-readable display
6. Provide clear visual feedback for both normal operation and error conditions
7. Maintain synchronous operation at 50 MHz clock frequency

2. Functional Specifications

Technical Specifications

- **Operand Size:** 4-bit inputs (0-15 decimal range)
- **Operation Set:** Addition, subtraction, multiplication, integer division
- **Clock Frequency:** 50 MHz operation
- **User Interface:** Toggle switches for inputs, push buttons for control
- **Display Output:** Eight seven-segment LEDs with sign indication
- ❖ SW(7:4) selects **Operand A** (4-bit signed input)
- ❖ SW(3:0) selects **Operand B** (4-bit signed input)
- ❖ SW(17:16) selects **Operation Type**:
 - 00 → Addition (A+B)

- 01 → Subtraction (A-B)
 - 10 → Multiplication (A-B)
 - 11 → Division (A/B)
- ❖ KEY(0) acts as the **calculation trigger**
 - ❖ KEY(3) provides a **reset signal**
 - ❖ Operands A and B are displayed on HEX7–HEX4 respectively
 - ❖ Result is shown on HEX2–HEX0
 - ❖ Negative numbers display a “–” sign on HEX7 or HEX5
 - ❖ Invalid (out-of-range) inputs display an “E” for error
 - ❖ LEDR(17:16) indicate the current operation type

3. Design Approach

Inputs:

Hardware Implementation

- **FPGA Board: DE2-115**
- **Input Devices:** 18 on-board slide switches (SW 17 down to 0)
 - sw (17:16): Operation selection
 - sw (7:4): Operand A
 - sw (3:0): Operand B
 - key (3:0): Push buttons (active low)
 - key (0): Calculate trigger
 - key (3): System reset
- **Output Devices:** 8 seven-segment displays (HEX7–HEX0), LEDs (LEDR, LEDG)
- **Clock Source:** 50 MHz on-board oscillator

VHDL Modules

- **Top-Level Entity (project1.vhd):**
Handles all logic including operand decoding, sign handling, arithmetic operations, and seven-segment display encoding.
- **Double-Dabble Converter (double_dabble_6bit.vhd):**
Converts 6-bit binary results into two BCD digits for display on HEX0–HEX1.
- **Testbench (project1_tb.vhd):**
Provides input stimulus to validate all operations via ModelSim simulation.

Software Tools

- **Intel Quartus Prime Lite 18.1** – Synthesis and hardware testing
- **ModelSim-Altera** – Functional simulation and waveform verification

4. Architecture Description

Top-Level Entity: project1

The main entity implements a structural architecture that integrates all calculator components:

entity project1 is

```
port(  
    clock_50 : in std_logic;  
    key      : in std_logic_vector(3 downto 0);  
    sw       : in std_logic_vector(17 downto 0);  
  
    hex0, hex1, hex2, hex3 : out std_logic_vector(6 downto 0);  
    hex4, hex5, hex6, hex7 : out std_logic_vector(6 downto 0);  
    ledr : out std_logic_vector(17 downto 0);  
    ledg : out std_logic_vector(8 downto 0)  
);
```

end entity project1;

Major Components

1. Input Processing Module

- ✓ Operand validation (0-9 range check)
- ✓ Sign detection and conversion to signed format
- ✓ Operation selection decoding

2. Arithmetic Logic Unit

- ✓ Synchronous operation using registered inputs
- ✓ Four arithmetic operations with proper result sizing
- ✓ Division-by-zero handling

3. Double Dabble Converter

- ✓ 6-bit binary to 8-bit BCD conversion
- ✓ Efficient shift-and-add-3 algorithm implementation

4. Display Controller

- ✓ Seven-segment LED decoding
- ✓ Sign indication for negative numbers
- ✓ Error display for invalid inputs

Double Dabble Algorithm Implementation

The double_dabble_6bit component implements the efficient binary-to-BCD conversion algorithm:

```

entity double_dabble_6bit is
  port(
    binary_in : in std_logic_vector(5 downto 0);
    bcd_out   : out std_logic_vector(7 downto 0)
  );
end entity double_dabble_6bit;

```

Algorithm Steps:

1. Initialize BCD registers to zero
2. For each bit in the binary input (MSB first):
 - Check each BCD digit: if ≥ 5 , add 3
 - Shift left entire BCD-binary combination
3. Output contains two BCD digits (tens and units)

Arithmetic Operations

The ALU performs operations on 4-bit signed inputs with 8-bit result precision:

- ✓ **Addition/Subtraction:** 8-bit result to handle overflow
- ✓ **Multiplication:** 8-bit result (4×4-bit multiplication)
- ✓ **Division:** Integer division with truncation
- ✓ **Error Handling:** Division by zero returns zero

5. Hardware and Software interaction and Executions

The synthesized design was successfully programmed onto the **DE2-115 FPGA board**.
Observed hardware behavior matched simulation:

Once the design is loaded, the calculator is ready to use.

1. **Provide Power:** Ensure the DE2-115 board is powered on.
2. **Set Operands:**
 - ✓ Use switches **SW3 - SW0** to set the 4-bit value for **Operand B**.
 - ✓ Use switches **SW7 - SW4** to set the 4-bit value for **Operand A**.
3. **Select Operation:**
 - ✓ Use switches **SW17** and **SW16** to choose the operation.
 - ✓ The corresponding red LEDs **LEDR17** and **LEDR16** will light up to show selection.
 - 00 = Addition

- 01 = Subtraction
- 10 = Multiplication
- 11 = Division

4. Execute Calculation:

- ✓ Press and release the **KEY0** button. This is your "equals" button.
- ✓ The system will calculate the result and display it.

5. Read the Display:

- ✓ **HEX7 & HEX6:** Display Operand A (with a negative sign if applicable).
- ✓ **HEX5 & HEX4:** Display Operand B (with a negative sign if applicable).
- ✓ **HEX2, HEX1, HEX0:** Display the Result. HEX2 shows a negative sign if the result is negative. HEX1 shows the tens digit, and HEX0 shows the ones digit.

6. Reset Functions:

- ✓ **KEY1 (Reset):** Press to perform a full system reset. This will clear the result and return the state machine to IDLE.
- ✓ **KEY3 (Display Reset):** Press to clear the displayed result to "00" without fully resetting the state machine.

Display Management

- **Hex7-Hex6:** Operand A with sign indication
- **Hex5-Hex4:** Operand B with sign indication
- **Hex2-Hex0:** Calculation result with sign
- **Hex3:** Unused (always off)

Input Validation

- ✓ Operands validated for BCD range (0 to 9)
- ✓ Invalid inputs display 'E' error code
- ✓ Ensures meaningful calculation results

So after all,

- Each operation produced accurate results for valid 4-bit signed inputs (0 to 9).
- Negative operands displayed correctly
- The board's LEDs reflected operation selection.
- The calculator responded to key presses for each computation cycle

6. Functional Block Diagram

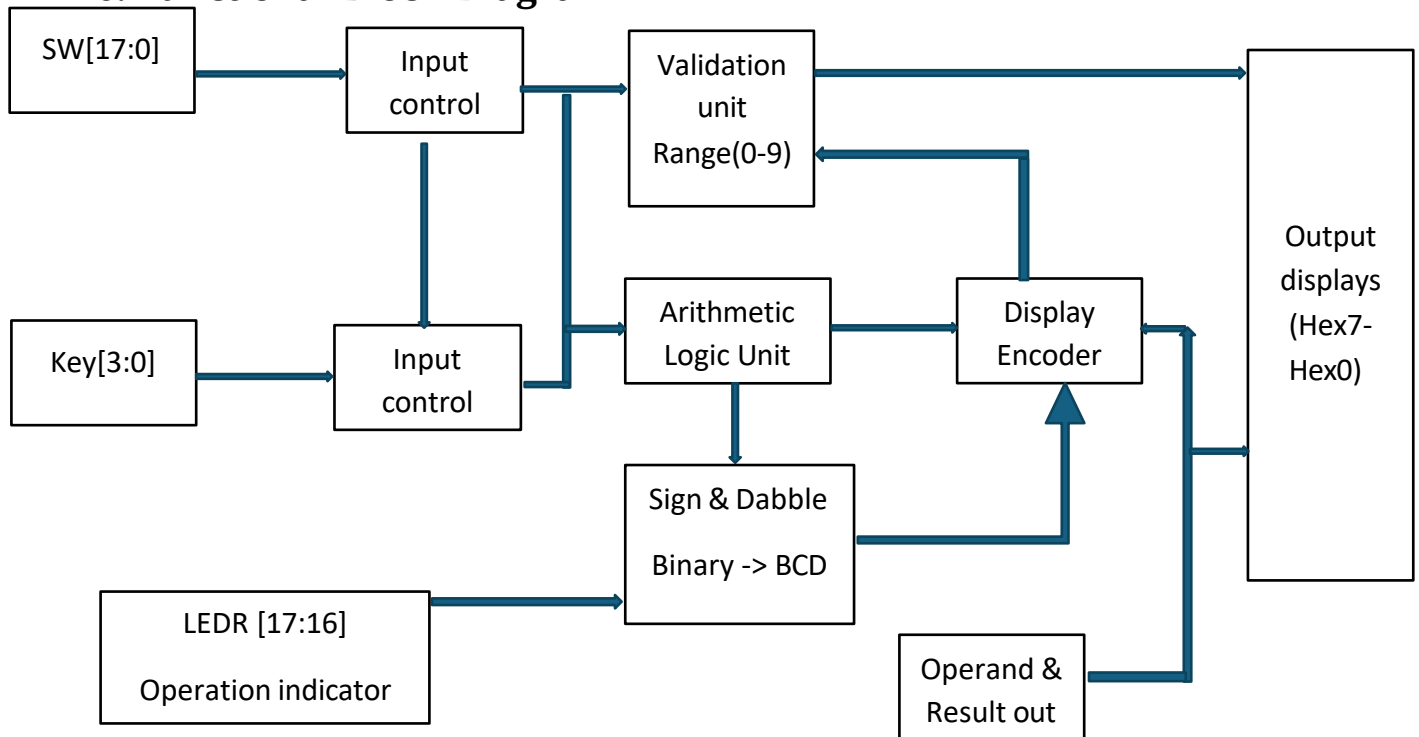


Figure 1: Functional block diagram of simple 4-bit calculator

7. Schematics/ Photos

I) Design Verification

Design was verified successfully using testbench files on Modelsim software as shown below.

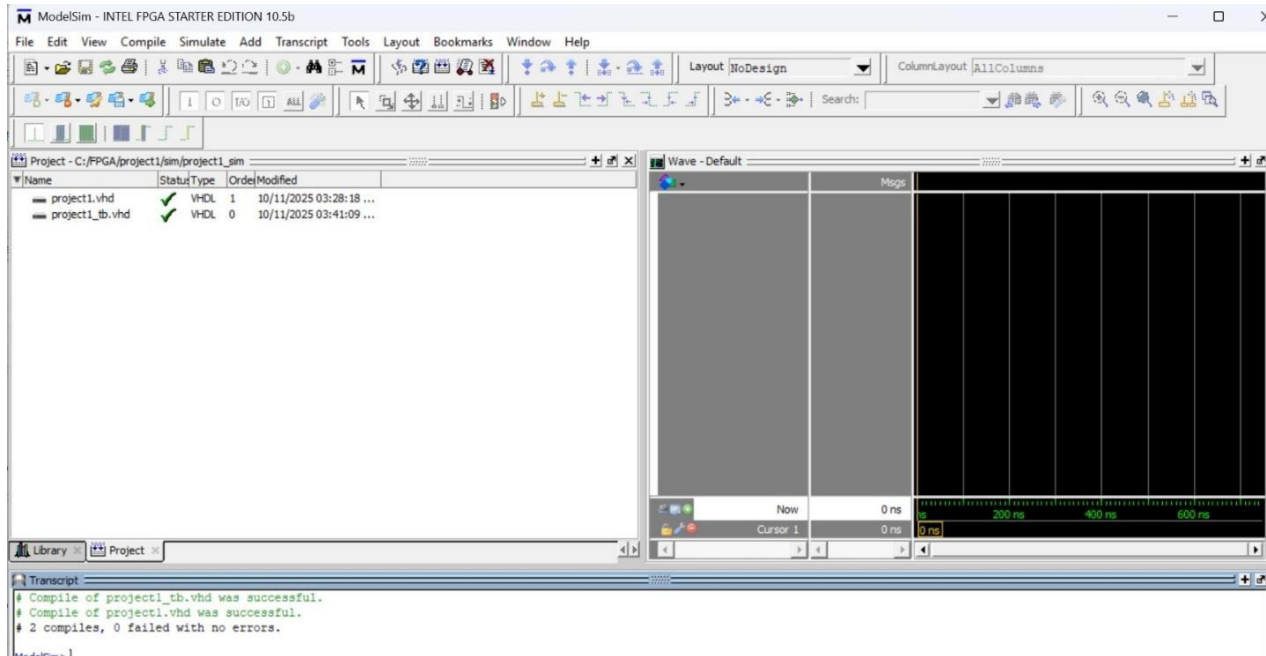


Figure 2: project1.vhd and project1-tb.vhd file compiled successfully

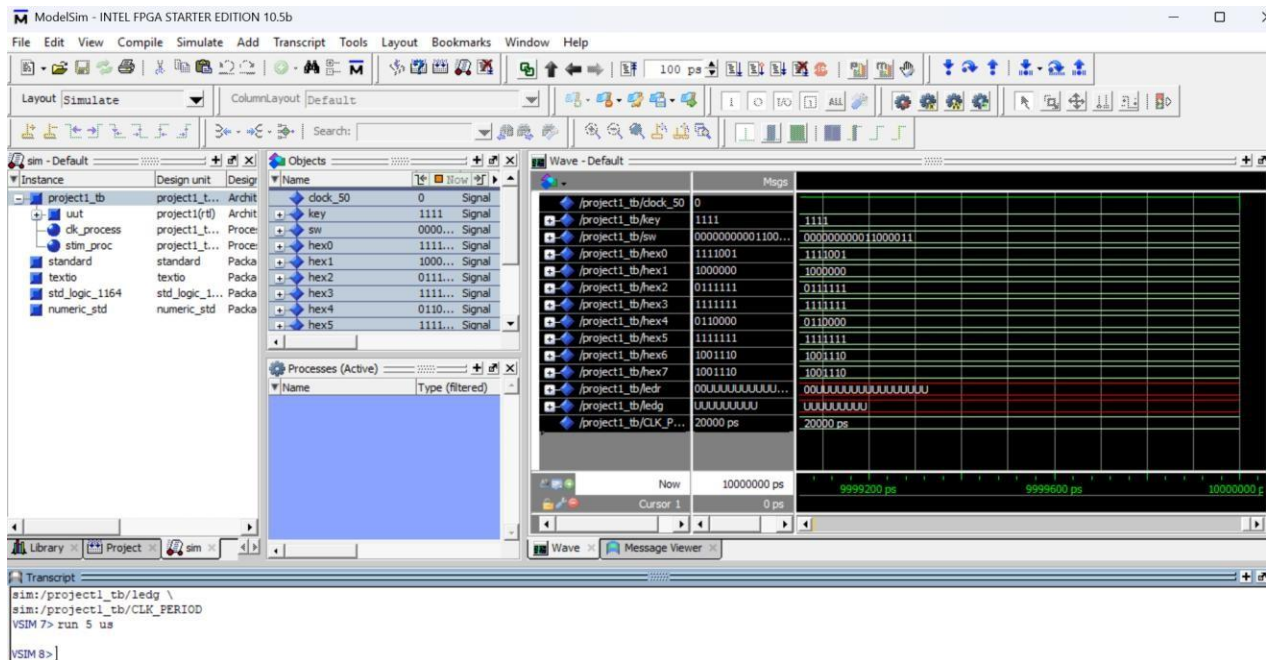


Figure 3: simulation waveform for simple 4-bit calculator

II) Performance of Arithmetic Operations on DE2-115 board

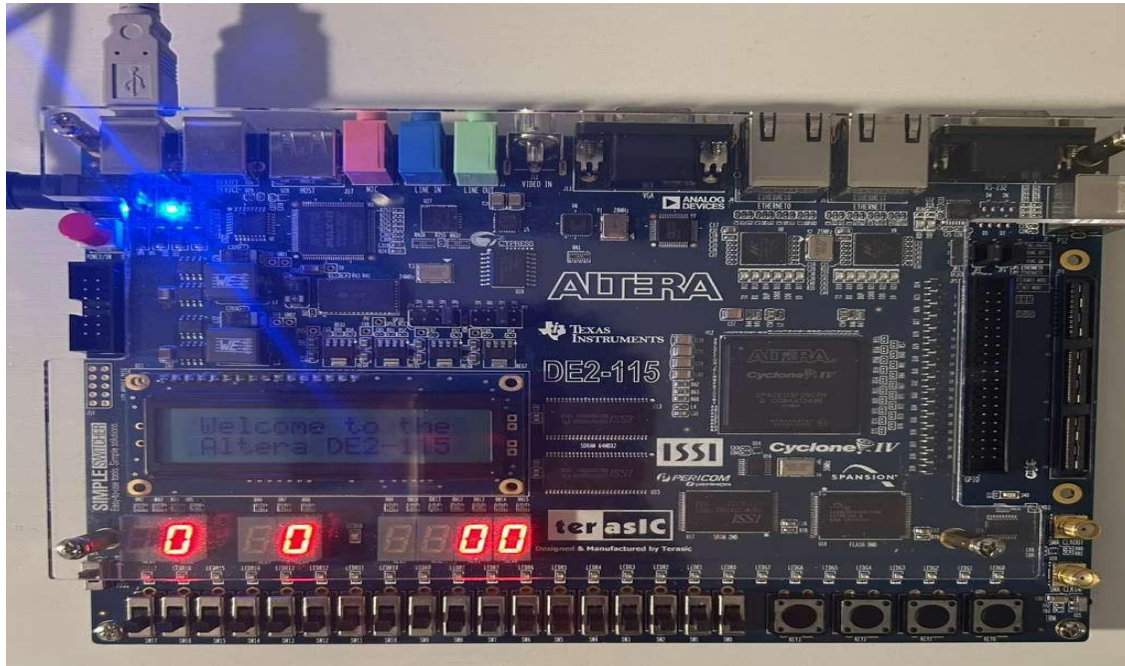


Figure 4: The Initial condition of hardware

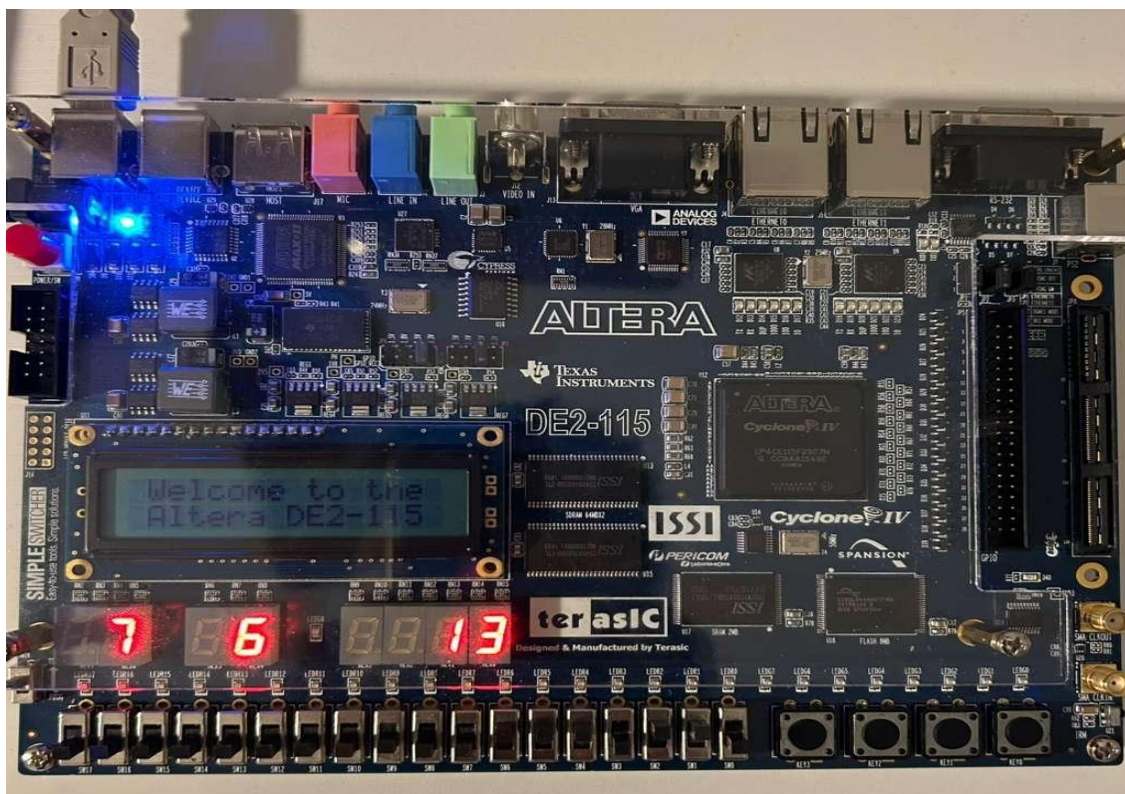


Figure 5: Addition

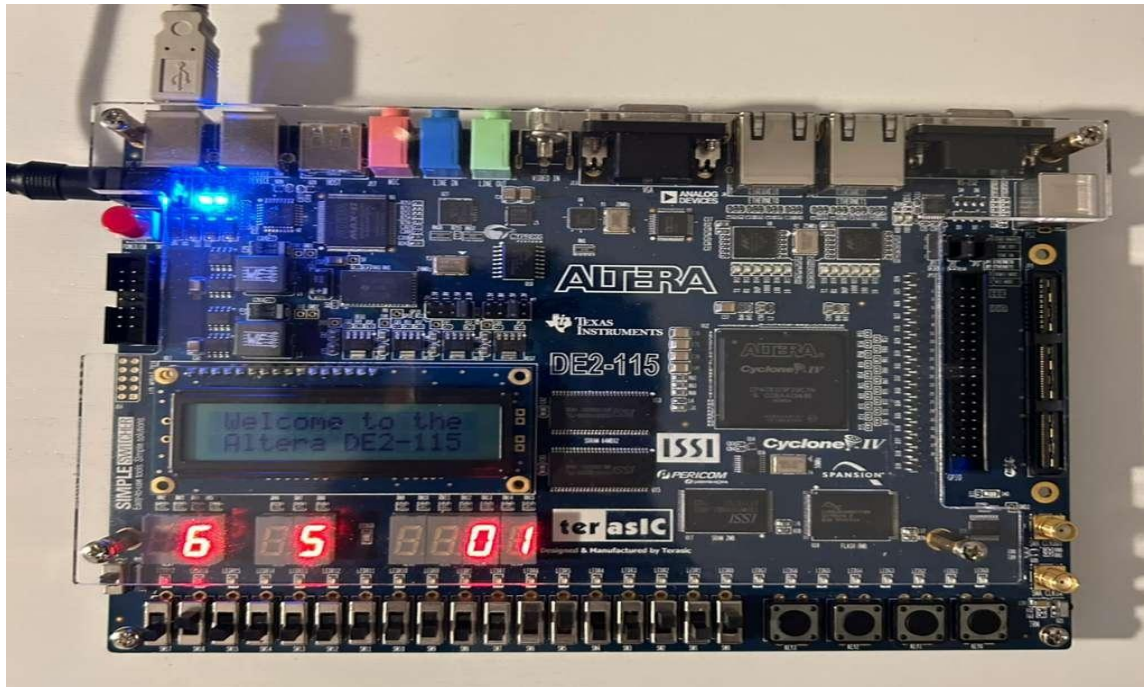


Figure 6: Subtraction

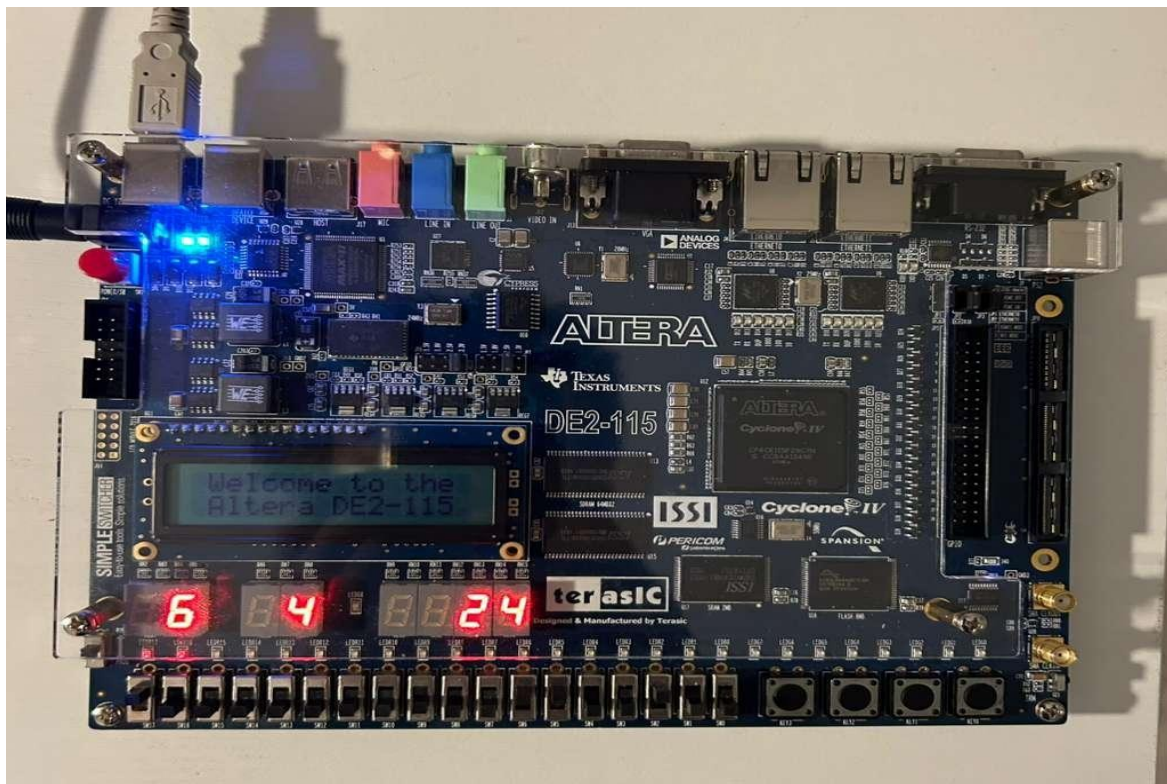


Figure 7: Multiplication

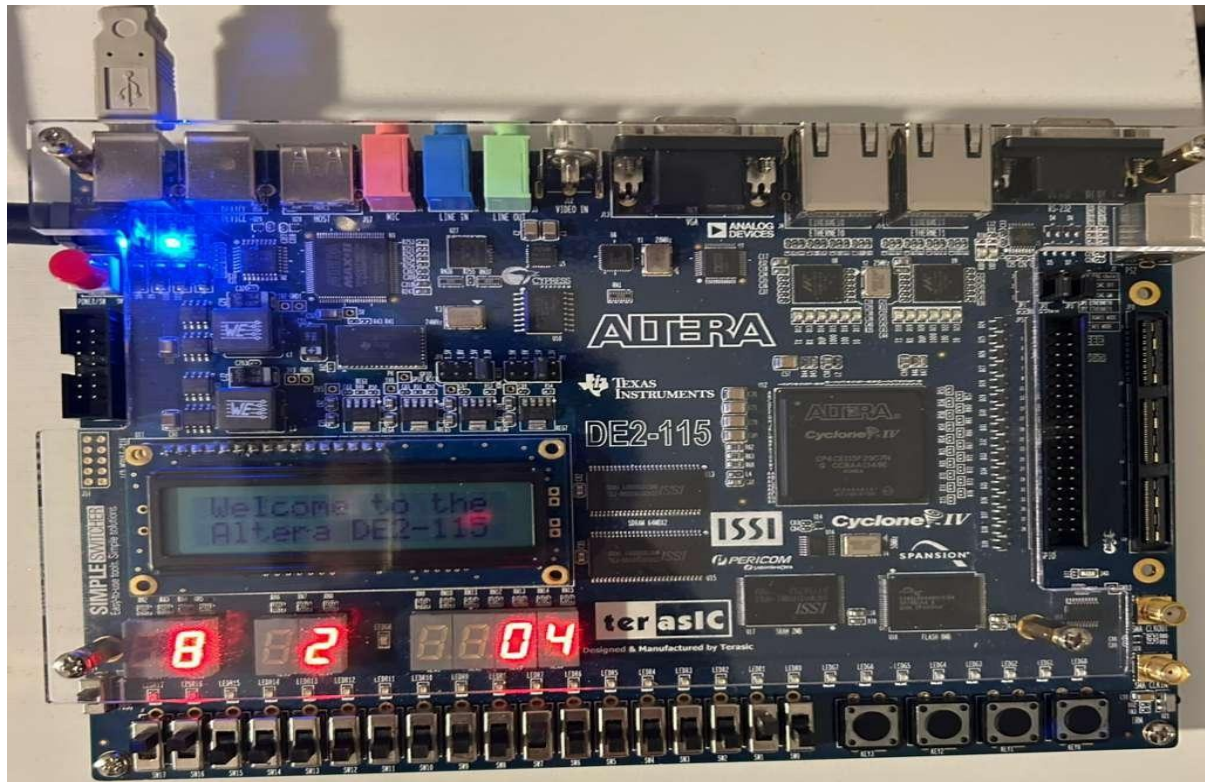


Figure 8: Division

8. Conclusions / Lessons Learned

The project successfully met all objectives of the assignment:

- Implemented a **working 4-bit signed calculator** on FPGA
- Validated functionality through both **simulation** and **hardware testing**
- Demonstrated effective **use of modular VHDL design**
- Gained experience in **testbench creation** and waveform debugging

Lessons

- Proper initialization of signals prevents simulation warnings.
- Understanding **signed vs unsigned conversions** in VHDL (`to_signed`, `to_integer`) is critical.
- Seven-segment encoding for both digits and signs must be handled carefully for clarity.
- Modular architecture improves readability and simplifies debugging.
- Simulation is indispensable for confirming correctness before synthesis.

9. Appendix / Source Code Listing

- `project1.vhd` – Top-level calculator
- `double_dabble_6bit.vhd` – Binary-to-BCD converter
- `project1_tb.vhd` – Testbench for functional verification